

PROJECTS

DAY 99
17/10/2025

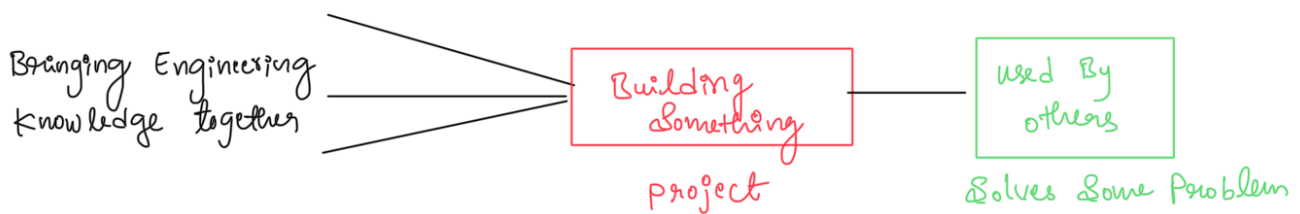
often you would have heard in the interview or internships Every Where they will ask Students what projects they would have done and asked to Explain there project ?

During 4 years of Engineering

We learn cs fundamentals, coding. But this are the foundational knowledge to become a Software Engineer

While doing projects We Bring all this computer fundamentals together and try to Build Something

Technically The term Software Engineering is using the knowledge of computer science, coding & Technology and Building Something which used By many People.



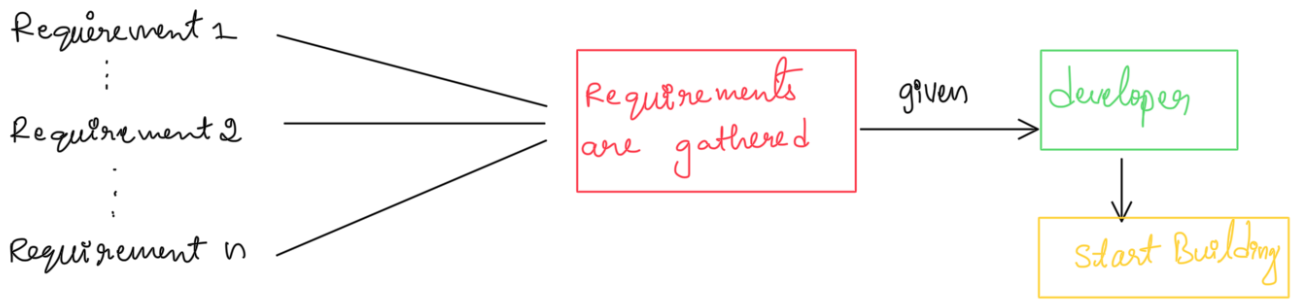
technically Engineer Have the Responsibility

you are trying to Build products or Services makes Humans to increase there productivity, or helps people to complete their tasks or Save Some time

What is project → project is Basically thought of Doing Something

phases of projects

① Requirement collection → trying to understand client needs



Requirement Collection Sounds Easy But its a Very Difficult thing to do. (Have to collect each and every things)

Bcz Before Building a project on Any Domain the Engineers Have to Understand it Really Well. (understanding the Domain)

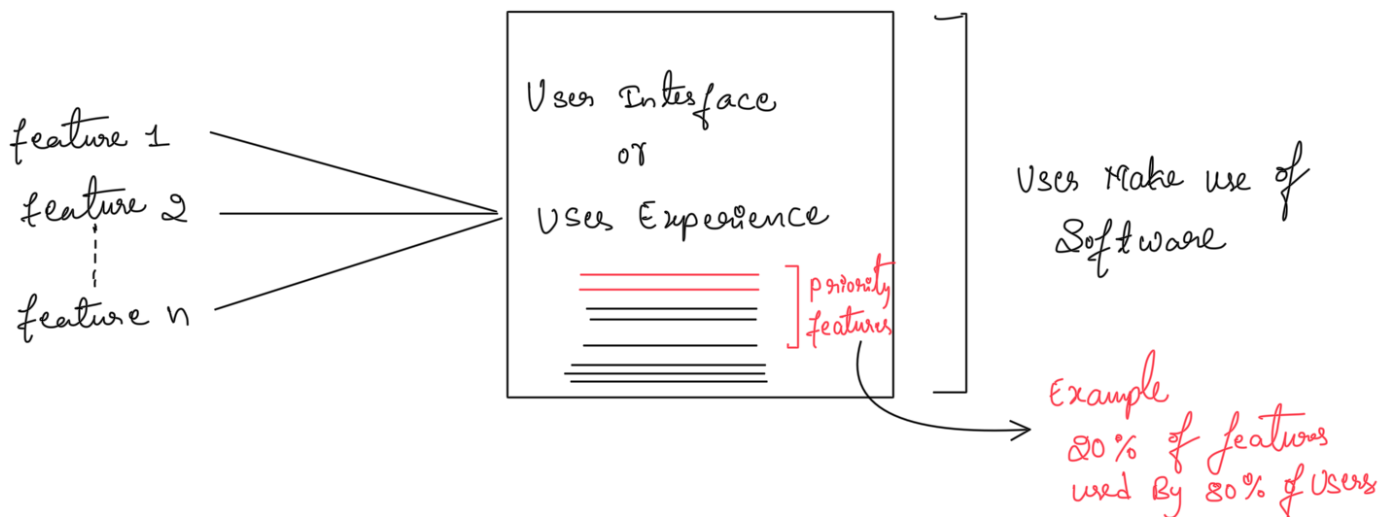
⑧ Adding Capabilities or features to the Software

Capability Means What can an Software do for a User What feature it have

⑨ User stories or User Experience

Making a story for Visualizing the Software features and Making Software in a such way that users can use it with good User Interface

and Setting priority for some features, The priority features are those features which are the Main thing of the Software

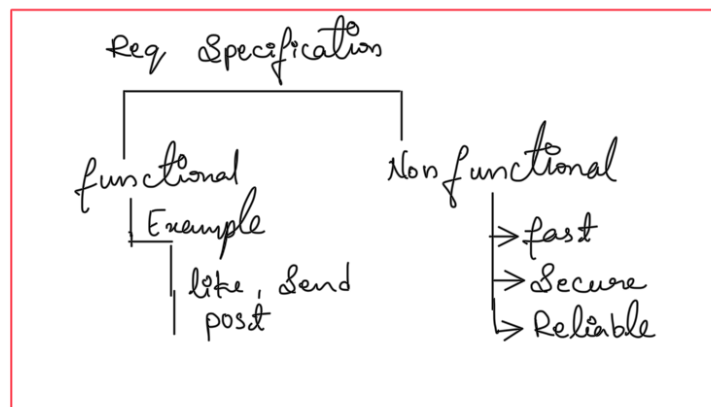


features priority

- ① Must do features (P₀ priority)
 - ② Must Have features (P₁ priority)
 - ③ Good to Have features (P₂ priority)
 - ④ Not Important features (P₃ priority)
- 

Doing All these Analysis and Writing it down in terms of features in terms of user stories, in terms of priority is called as Requirement analysis.

③ Documentation → Requirement Specification



A good Software Requirement Specification lead to a good product, a Better product, polished product and less Rework between development and testing. Can Actually Reduce the Development time.

1 What is a Software Project?

A software project is a planned effort to **design, develop, and deliver** a software application or system that solves a specific user or business problem.

Example:

It includes:

Building an "AI Weather Assistant App" that fetches real-time weather, predicts patterns, and gives voice responses.

- Objectives (what we're solving)
- Scope (features, boundaries)
- Resources (developers, designers, tools)
- Timeline (phases & milestones)
- Deliverables (working software, docs, reports)

🔧 2 What is Software Project Development?

It's the process of turning an idea into a working product through structured stages, tools, and collaboration.

🔄 Typical Phases (Software Development Life Cycle — SDLC)

Phase	Description	Example
Requirement Gathering	Understand what the user needs	"The user wants daily weather updates by voice."
System Design	Create architecture & modules	Frontend (UI), Backend (API), Database
Implementation (Coding)	Write and integrate code	Using Java + Spring Boot
Testing	Verify correctness and reliability	Unit, Integration, User testing
Deployment	Release software to users	Deploy on AWS or local server
Maintenance	Fix bugs, add features	Version updates, bug fixes

👤 3 Responsibilities of a Software Developer

A software developer's role goes beyond just writing code — you're solving *real-world problems* with logic and design.

Responsibility	Description
Requirement Analysis	Understand what needs to be built
Design and Architecture	Structure the code & choose patterns
Implementation	Write efficient, clean, reusable code
Testing & Debugging	Ensure quality and reliability
Documentation	Comment code, update project wiki
Collaboration	Work with designers, PMs, testers
Continuous Learning	Keep up with tools, APIs, new tech
Version Control	Use Git/GitHub to track progress

🧠 Real-world analogy:

A developer is like an *engineer* in a construction project — you read the blueprints (requirements), use the right tools (code, APIs), and ensure the final building (software) works as expected.

📋 4 Requirement Collection (and its related concepts)

Requirement collection is the first and most critical phase — because everything in the project depends on *what you build* and *why*.

💡 What It Involves

Step	Purpose	Example
Gather Requirements	Talk to stakeholders & users	"App should send daily reminders."
Identify Capabilities	What can the system do?	"Fetch weather data from an API."
Define Features	Break down into functionalities	"Voice input", "Temperature display."
Set Priorities	Must-have vs nice-to-have	Must: "API integration"; Later: "Voice output"
Write User Stories	Short descriptions from user view	"As a user, I want to check weather by voice"
Design User Experience (UX)	How users interact	Buttons, screens, flow, accessibility

🌱 5 How Requirement Collection Helps

Benefit	Explanation
✅ Clear Understanding	Avoid confusion later — all stakeholders aligned

Scope Management

Prevents “feature creep” (unnecessary additions)

Time Estimation

Helps plan milestones & allocate resources

Design Accuracy

Developers know what features to build

Better Testing

QA knows what “correct behavior” means

In simple words:

Good requirements are like a **GPS** for developers — without them, you’ll get lost even with great tools.

6 Documentation & Requirement Specification

Software Requirement Specification (SRS)

An **SRS document** describes *what* the system will do, *not how* to build it.

It’s the **official agreement** between developers and clients.

7 Responsibilities of a Project Manager

A **Project Manager (PM)** bridges the gap between technical and business sides.

Task	Description
Planning	Define goals, timelines, deliverables
Team Coordination	Assign work to developers, testers, designers
Resource Allocation	Manage budget, time, and manpower
Stakeholder Communication	Keep clients updated
Risk Management	Identify and mitigate problems early
Documentation Oversight	Ensure clear SRS, reports, and progress tracking
Monitoring Progress	Track deadlines and quality using Agile tools (like Jira, Trello, Asana)

Analogy:

A Project Manager is like a **conductor of an orchestra** — not playing instruments, but making sure every musician (developer, tester, designer) plays in harmony.

Summary Table

Role/Concept	Purpose	Real-life Analogy
Software Project	Goal-driven software development	Building a bridge
Software Development	Process from idea → working product	Construction stages
Developer	Builds and maintains code	Engineer
Requirement Collection	Understand what to build	Talking to client
SRS Document	Agreement on features and scope	Blueprint
Project Manager	Oversees project progress and quality	Project supervisor

Structure of an SRS:

1. Introduction

→ Project purpose, scope, users

2. Overall Description

→ System overview, features, constraints

3. Functional Requirements

→ Each function or behavior clearly defined

4. Non-functional Requirements

→ Performance, security, scalability

5. Use Cases & User Stories

→ Step-by-step scenarios

6. Acceptance Criteria

→ When do we consider it “done”?